

Reverse file APK cho người mới bắt đầu

Từ cấu trúc Android và DEX đến phân tích tĩnh, phân tích động, Smali, JNI và quy trình làm lab

Chuỗi tạo APK và điểm reverse



Reverse đi ngược từ gói APK về cấu trúc, bytecode, tài nguyên và hành vi gần với mã nguồn - nhưng không khôi phục hoàn hảo source ban đầu.

Đối tượng đọc

Người đã biết lập trình cơ bản và reverse Windows ở mức nhập môn, nhưng chưa từng mở hoặc phân tích một APK.

Phiên bản tài liệu 1.0 | Cập nhật tháng 9 năm 2026

Phạm vi sử dụng. Chỉ phân tích ứng dụng do bạn sở hữu, ứng dụng mã nguồn mở, crackme/lab hoặc mục tiêu được cấp phép. Không dùng tài liệu để vượt quyền truy cập, đánh cắp dữ liệu, gian lận thanh toán hay phát tán bản sửa đổi trái phép.

Cách dùng tài liệu này

Tài liệu không giả định bạn đã học Android. Mục tiêu đầu tiên không phải là thuộc mọi API, mà là xây được một mô hình đúng: APK chứa gì, Android gọi code từ đâu, công cụ đang biến đổi dữ liệu nào và bằng chứng nào đủ mạnh để kết luận. Sau khi đọc hết phần nền tảng, bạn có thể lấy một APK lab, lập bản đồ component, lần theo một chức năng từ giao diện đến code và kiểm chứng nó khi chương trình chạy.

Đừng đọc tuyến tính một app lớn từ class đầu tiên đến class cuối cùng. Cách hiệu quả là đi từ sự kiện quan sát được đến entry point, từ entry point đến data flow, rồi dùng runtime để xác nhận. Đây cũng là cách tránh bị ngợp khi gặp hàng nghìn class thư viện và tên bị obfuscate.

Kết quả đầu ra sau khi học

- Giải thích được APK, AAB, DEX, ART, Smali, resources.arsc, Manifest, chữ ký và thư viện native liên hệ với nhau thế nào.
- Cài được bộ công cụ tối thiểu trên Windows và tạo emulator riêng cho phân tích.
- Triage một APK, xác định package, certificate, component, quyền, ABI và bề mặt đáng chú ý.
- Dùng JADX để đọc code gần Java, dùng Apktool để đọc resource và Smali, biết lúc nào phải kiểm tra bytecode thay vì tin decompiler.
- Theo dõi một hành vi bằng adb, logcat, proxy và Frida trên lab được phép.
- Sửa một APK lab đơn giản, build, zipalign, ký lại và hiểu vì sao chữ ký gốc không còn hợp lệ.
- Nhận diện obfuscation, reflection, dynamic loading, JNI và các cơ chế anti-analysis thường gặp.

Mục lục

- 1 Mô hình Android tối thiểu cho người reverse
- 2 APK được tạo ra như thế nào
- 3 Giải phẫu một file APK
- 4 DEX ART Smali và giới hạn của decompiler
- 5 Chuẩn bị môi trường lab trên Windows
- 6 Quy trình phân tích chuẩn
- 7 Phân tích tĩnh bằng JADX và Apktool
- 8 Phân tích động bằng adb logcat proxy và Frida
- 9 Native code JNI và file so
- 10 Sửa build và ký lại APK lab
- 11 Obfuscation packing và anti-analysis
- 12 Bài thực hành từ đầu đến cuối
- 13 Lộ trình học sáu tuần
- 14 Cheat sheet xử lý lỗi và tài liệu tham khảo

1 Mô hình Android tối thiểu cho người reverse

Các lớp cần liên hệ khi reverse APK

Ứng dụng	Activity Service Receiver Provider
Android Framework	Intent Binder Permission API
ART và thư viện native	DEX GC JNI libc SSL
Linux Kernel	process UID file network driver

Phần lớn app nằm ở DEX; logic nhạy cảm có thể chuyển xuống JNI và thư viện ELF trong lib/<ABI>.

1.1 App Android không có một hàm main duy nhất

Trong chương trình console Windows, ta thường bắt đầu từ main hoặc WinMain. Android tổ chức ứng dụng theo component và lifecycle. Hệ điều hành có thể tạo process khi cần, khởi tạo một Activity khi người dùng mở màn hình, gọi BroadcastReceiver khi có sự kiện, hoặc bind tới Service từ một process khác. Vì vậy câu hỏi đúng không phải là main ở đâu, mà là hành vi này đi vào app qua component hoặc callback nào.

Component	Vai trò	Điểm reverse thường gặp
Activity	Một màn hình hoặc luồng tương tác	onCreate, callback nút bấm, Intent nhận vào
Service	Công việc nền hoặc API bind	onStartCommand, onBind, foreground service
BroadcastReceiver	Nhận sự kiện hệ thống hoặc app	onReceive, action trong intent-filter
ContentProvider	Cung cấp dữ liệu qua URI	query, insert, update, delete, quyền truy cập

Mẹo định hướng. Khi thấy một hành vi trên màn hình, hãy tìm Activity hiện tại, layout hoặc Compose screen, ID của view, callback, rồi mới lần tới logic xử lý.

1.2 Process UID sandbox và quyền

Mỗi app thường được Android cấp một Linux UID riêng. File riêng trong `/data/data/<package>` hoặc `/data/user/0/<package>` chỉ UID đó và tiến trình có đặc quyền phù hợp mới đọc được. Permission cho phép app gọi một số API hoặc tương tác với dữ liệu/component; nó không tự động làm code an toàn. Một exported component cấu hình sai vẫn có thể tạo bề mặt tấn công dù app yêu cầu nhiều quyền.

- **UID và sandbox:** giải thích vì sao app bình thường không đọc trực tiếp private data của app khác.
- **Permission:** cơ chế Android dùng để kiểm soát thao tác nhạy cảm và giao tiếp giữa app.
- **Binder IPC:** cơ chế gọi qua process; nhiều API framework cuối cùng đi qua Binder tới system service.
- **SELinux:** lớp chính sách bổ sung, đặc biệt quan trọng khi làm việc với thiết bị root và service hệ thống.

1.3 Intent lifecycle và callback

Intent là thông điệp yêu cầu Android kích hoạt component hoặc thực hiện hành động. Intent explicit nêu component đích; implicit mô tả action, data URI, category và để hệ thống tìm component phù hợp. Khi reverse, extras, URI và flags của Intent thường là dữ liệu đầu vào quan trọng. Lifecycle lại giải thích tại sao code chạy nhiều lần, chạy sau khi xoay màn hình, hoặc chạy dù bạn không thấy lệnh gọi trực tiếp trong app.

Mẫu Java gần đúng do decompiler tạo ra

```
// Ví dụ dữ liệu cần lần theo trong code decompile
Intent i = getIntent();
String token = i.getStringExtra("token");
Uri deepLink = i.getData();
```

1.4 Java Kotlin và C++ cùng tồn tại

Logic app thường được biên dịch thành DEX từ Java hoặc Kotlin. C/C++ được NDK biên dịch thành thư viện ELF .so theo từng ABI và được gọi qua JNI. Flutter, React Native, Unity và các framework đa nền tảng thay đổi nơi logic chính nằm: Flutter có libapp.so và Dart snapshot; Unity có Mono hoặc IL2CPP; React Native có JavaScript bundle; ứng dụng WebView có thể để phần lớn logic trong HTML/JavaScript. Bước triage phải nhận diện stack trước khi chọn công cụ.

2 APK được tạo ra như thế nào

2.1 Chuỗi build và ý nghĩa khi đi ngược

Gradle phối hợp compiler, Android Gradle Plugin và build tools. Java/Kotlin được chuyển thành bytecode trung gian rồi D8 tạo DEX; R8 có thể shrink, optimize và obfuscate. Resource được AAPT2 biên dịch, link thành resources.arsc và XML nhị phân. Native code được compiler/linker tạo thành .so. Cuối cùng công cụ đóng gói, zipalign và ký số tạo APK có thể cài.

Khi build	Khi reverse	Thông tin có thể đã mất
Java/Kotlin -> DEX	DEX -> Java giả lập hoặc Smali	comment, tên local, cấu trúc source chính xác
Resource -> bảng ID	resources.arsc -> XML và tên resource	một phần ngữ cảnh build
C/C++ -> ELF .so	disassembly/decompile native	type, tên hàm bị strip, macro, template
R8 optimize/obfuscate	đọc control flow sau biến đổi	tên class/method và ranh giới hàm

Kết luận quan trọng. Decompiler tạo biểu diễn dễ đọc, không khôi phục source gốc. Một đoạn Java đẹp vẫn chỉ là giả thuyết về bytecode.

2.2 APK và AAB khác nhau

APK là gói cài đặt cho thiết bị. AAB là định dạng phát hành chứa module và metadata; Google Play hoặc bundletool sinh ra APK tối ưu theo thiết bị. Vì vậy file kéo từ thiết bị có thể chỉ là base APK cộng nhiều split APK cho ABI, ngôn ngữ hoặc mật độ màn hình. Nếu chỉ mở base.apk, bạn có thể thiếu resource hoặc native library cần thiết.

Dạng	Có cài trực tiếp	Điều cần lưu ý
APK đơn	Có	Một file thường chứa đủ code và resource
AAB	Không	Dùng bundletool hoặc dịch vụ phân phối để sinh APK
Split APK	Cài theo bộ	base.apk đi cùng config splits hoặc feature modules
XAPK APKM APKS	Tùy công cụ	Thường là container chứa nhiều APK và metadata

3 Giải phẫu một file APK

APK dùng container ZIP nhưng không phải chỉ đổi đuôi thành .zip là hiểu được toàn bộ. Bạn có thể liệt kê và trích file bằng công cụ ZIP, nhưng Manifest và nhiều XML/resource đã ở dạng biên dịch. JADX, Apktool và aapt2 hiểu các định dạng Android này.

Đường dẫn	Nội dung	Giá trị khi reverse
AndroidManifest.xml	Component, permission, SDK, cấu hình app	Điểm vào và bề mặt giao tiếp
classes.dex classes2.dex	Dalvik bytecode	Logic Java/Kotlin
resources.arsc	Bảng resource đã biên dịch	Ánh xạ ID sang tên và giá trị
res/	Layout, drawable, XML cấu hình	UI, network config, FileProvider paths
assets/	File app đọc nguyên trạng	JSON, model, JS, database, payload
lib/<ABI>/*.so	ELF native library	JNI, crypto, engine, anti-analysis
META-INF/	Metadata và phần ký kiểu JAR	Hữu ích với v1 signing; không phản ánh toàn bộ v2/v3
kotlin/ và metadata	Thông tin Kotlin hoặc module	Gợi ý ngôn ngữ và cấu trúc

3.1 AndroidManifest là bản đồ xuất phát

- Package name, version, minSdk, targetSdk và application class.
- Activity có MAIN và LAUNCHER; deep link có VIEW, BROWSABLE và data scheme/host/path.
- Component exported hoặc có intent-filter; permission bảo vệ component.
- debuggable, allowBackup, usesCleartextTraffic, networkSecurityConfig và testOnly.
- Provider authority, grantUriPermissions, FileProvider paths và metadata.
- Tên service, receiver, instrumentation và custom permission.

Không kết luận từ một cờ. Ví dụ exported=true không tự động là lỗ hổng. Cần xác định dữ liệu nào component nhận, kiểm tra caller ra sao và hành động nào xảy ra.

3.2 Chữ ký APK

Android dùng certificate để nhận diện tác giả/ khóa ký và quyết định khả năng update, chia sẻ quyền hoặc thiết lập trust. Khi sửa bất kỳ byte nào, chữ ký ban đầu không còn hợp lệ. Bạn có thể ký bản lab bằng khóa của mình, nhưng nó không trở thành bản chính chủ và thường không thể cài đề lên bản ký bằng khóa khác.

Kiểm tra chữ ký và certificate

```
apksigner verify --verbose --print-certs app.apk
keytool -printcert -jarfile app.apk
```

APK Signature Scheme v1 bảo vệ entry theo mô hình JAR; v2 và v3 bảo vệ toàn bộ gói qua signing block; v4 hỗ trợ luồng cài đặt. Công cụ apksigner trong Android SDK Build Tools nên được dùng để verify thay vì chỉ nhìn META-INF.

4 DEX ART Smali và giới hạn của decompiler

4.1 DEX là bytecode hướng thanh ghi

DEX chứa class, field, method, string, type và instruction cho Android Runtime. Khác JVM bytecode thiên về stack, Dalvik bytecode dùng virtual register. Trong Smali, p0, p1 thường biểu diễn tham số và v0, v1 là local register theo quy ước hiển thị. Method instance có p0 là this.

Ví dụ Smali tối giản

```
.method private checkPin(Ljava/lang/String;)Z
.locals 2
const-string v0, "2468"
invoke-virtual {v0, p1}, Ljava/lang/String;.>equals(Ljava/lang/Object;)Z
move-result v1
return v1
.end method
```

Instruction	Ý nghĩa gần đúng
const-string v0, "x"	Đưa reference chuỗi vào v0
invoke-virtual {v0,p1}, ...	Gọi virtual method với các register làm đối số
move-result v1	Lấy giá trị trả về của lệnh invoke ngay trước
if-eqz v1, :label	Nhảy nếu v1 bằng 0 hoặc null tùy kiểu
iget/iput	Đọc hoặc ghi instance field
sget/sput	Đọc hoặc ghi static field

4.2 Java decompile và Smali phục vụ hai mục tiêu khác nhau

JADX rất tốt để hiểu nhanh cấu trúc, search string, find usage và dựng data flow. Smali sát DEX hơn, phù hợp khi decompiler báo lỗi, control flow kỳ lạ, cần xác nhận kiểu dữ liệu, hoặc sửa bytecode nhỏ. Đọc Java trước để định hướng; kiểm tra Smali ở điểm quyết định quan trọng.

Tình huống	Nên xem trước
Tìm chức năng đăng nhập nằm đâu	JADX Java view, strings, find usage
JADX hiện code không compile được	Smali hoặc chế độ fallback/simple
Tên class a b c	Manifest, resource ID, API calls, runtime trace
Cần đổi một điều kiện boolean trong lab	Smali quanh branch và return
Logic nằm trong native method	JNI mapping rồi Ghidra/IDA

4.3 Các dạng sai lệch decompiler

- Biến local và tên tham số được đoán hoặc sinh lại; tên có vẻ hợp lý không có nghĩa là tên gốc.
- Kotlin coroutine, suspend, inline lambda và state machine có thể trông rất rối.
- R8 có thể inline method, gộp branch, loại code chết hoặc thay đổi cấu trúc class.
- Try catch, synchronized, switch trên string và generic có thể được tái cấu trúc khác source.
- Decompiler có thể bỏ qua method lỗi hoặc chèn comment báo inconsistent code.

Thứ tự độ tin cậy. Hành vi quan sát được và dữ liệu runtime > DEX/Smali > decompiler output > phỏng đoán dựa trên tên.

5 Chuẩn bị môi trường lab trên Windows

5.1 Bộ công cụ tối thiểu

Công cụ	Mục đích	Mức cần thiết
Android Studio SDK	Emulator, SDK Manager và nền tảng Android	Bắt buộc cho lab thuận tiện
Platform Tools	adb và fastboot	Bắt buộc
Build Tools	aapt2, apksigner, zipalign	Bắt buộc khi kiểm tra/build
JADX GUI	DEX/APK sang Java gần đúng, search và xref	Bắt buộc
Apktool	Decode resource/Manifest và Smali, rebuild APK	Bắt buộc
Java JDK 64 bit	Chạy công cụ Java	Bắt buộc
Frida tools	Dynamic instrumentation	Học sau static
MobSF	Tự động hóa triage và báo cáo	Tùy chọn, không thay phân tích tay
Ghidra hoặc IDA	Phân tích thư viện native .so	Khi có native code

5.2 Cấu trúc thư mục lab

Gợi ý tổ chức dữ liệu

```
apk-lab/  
  samples/    # APK gốc, chỉ đọc  
  work/       # file giải nén và dự án Apktool  
  notes/      # nhật ký, giả thuyết, ảnh chụp  
  scripts/    # Frida script và helper
```

```
output/      # APK lab đã build/ký  
hashes.txt
```

Luôn giữ APK gốc ở chế độ chỉ đọc và ghi SHA-256 trước khi làm. Tạo snapshot emulator trước mỗi lab. Không đăng nhập tài khoản thật, không đồng bộ danh bạ thật và không để emulator dùng chung dữ liệu với môi trường cá nhân.

5.3 Kiểm tra cài đặt

Kiểm tra PATH và kết nối

```
adb version  
adb devices -l  
java -version  
jadx --version  
apktool --version  
apksigner version
```

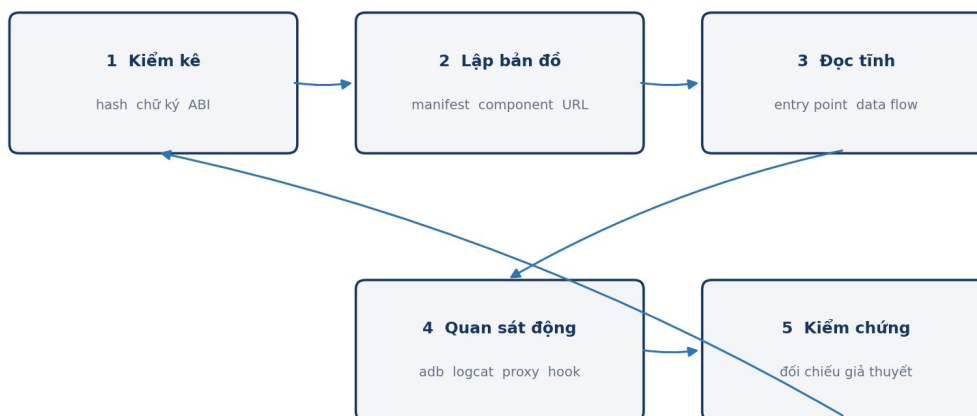
Windows. Nếu có nhiều Android SDK, dùng đường dẫn đầy đủ hoặc kiểm tra where adb để tránh chạy nhầm phiên bản. Đừng tải APK hoặc tool từ mirror không rõ nguồn.

5.4 Chọn target học tập

- **Bắt đầu:** một app tự viết có nút bấm, SharedPreferences và request HTTP giả lập.
- **Tiếp theo:** OWASP MAS Crackmes hoặc UnCrackable Apps, DIVA, InsecureShop và các app cố ý dễ tổn thương.
- **Sau đó:** app mã nguồn mở; reverse trước rồi đối chiếu source để đo sai lệch.
- **Tránh lúc đầu:** banking, game online, DRM thương mại, packer nặng, Flutter/Unity lớn hoặc app phụ thuộc server phức tạp.

6 Quy trình phân tích chuẩn

Quy trình phân tích theo vòng lặp bằng chứng



Khi kết quả động mâu thuẫn với decompiler, ưu tiên bytecode, dữ liệu runtime và thử nghiệm có kiểm soát.

6.1 Bước 0 xác định câu hỏi

Không mở JADX rồi đọc ngẫu nhiên. Viết câu hỏi có thể kiểm chứng: PIN được kiểm tra local hay gửi server; token được lưu ở đâu; deep link nào mở màn hình thanh toán; chuỗi được giải mã lúc nào; hàm native nào trả về trạng thái license trong crackme. Một câu hỏi hẹp giúp chọn đúng entry point và bằng chứng.

6.2 Bước 1 bảo toàn và kiểm kê

Triage nhanh trên Windows

```
certutil -hashfile app.apk SHA256
apksigner verify --verbose --print-certs app.apk
aapt2 dump badging app.apk
unzip -l app.apk
```

- Ghi tên file, kích thước, SHA-256, package, versionCode/versionName và thời điểm lấy mẫu.
- Ghi signer certificate digest; không đồng nhất certificate subject với danh tính pháp lý nếu chưa xác minh.
- Đếm classes*.dex; liệt kê ABI trong lib; tìm dấu hiệu Flutter, Unity, React Native hoặc packer.
- Xác định APK đơn hay một phần của split package.

6.3 Bước 2 lập bản đồ

Từ Manifest, tạo danh sách component và đánh dấu launcher, exported, deep link, permission và application class. Từ resource, liệt kê string đáng chú ý, endpoint, tên database, preference key. Từ code, đánh dấu package do tác giả viết và tách nó khỏi thư viện bên thứ ba.

Dấu mốc	Cách tìm	Câu hỏi tiếp theo
Application class	android:name trong application	Khởi tạo SDK, anti-analysis hay native library nào
Launcher Activity	MAIN + LAUNCHER	Màn hình đầu và navigation đi đâu
Deep link	VIEW + BROWSABLE + data	Input URI được validate ra sao
Native load	System.loadLibrary	Tên .so và JNI method liên quan
Network client	OkHttp Retrofit URL URLConnection	Base URL, interceptor, certificate pinning

6.4 Bước 3 lần data flow

Dùng mô hình Source -> Transform -> Sink. Source là dữ liệu vào: Intent extra, EditText, QR, file, response mạng. Transform là parse, decode, hash, decrypt hoặc validate. Sink là request mạng, database, log, WebView, command, file hoặc API nhạy cảm. Đừng chỉ search từ khóa secret; hãy theo đường đi của dữ liệu.

Ví dụ ghi chú data flow - chưa đủ để kết luận lỗ hổng

```
Source: getIntent().getStringExtra("redirect")
-> Uri.parse(value)
-> chỉ kiểm tra contains("trusted.com")
Sink: webView.loadUrl(uri.toString())
```

6.5 Bước 4 kiểm chứng động

Chọn quan sát tối thiểu có thể phân biệt các giả thuyết. Nếu chưa biết check nằm local hay server, tắt mạng và thử; quan sát log và traffic; hook method nghi ngờ để ghi tham số/return trên lab. Mỗi thử nghiệm chỉ thay một biến để kết quả có ý nghĩa.

6.6 Nhật ký phân tích

Thời điểm	Quan sát	Giả thuyết	Thử nghiệm	Kết quả
10:20	Nút Login gọi a.b.c()	c() tạo request	Hook c() và tắt mạng	Có call nhưng lỗi timeout
10:35	Response có field ok	UI tin field ok	Đổi response trong lab	Màn hình chuyển khi ok=true

7 Phân tích tĩnh bằng JADX và Apktool

7.1 Triage trong JADX GUI

- Mở APK và chờ index xong; xem warnings để biết có method decompile lỗi.
- Đọc Manifest và Resources trước; tìm package ứng dụng, không bắt đầu trong androidx, kotlin hay com.google.
- Search text cho URL, scheme, error message, key preference, database name, native library và API nhạy cảm.
- Find usage từ string/resource ID đến callback; rename alias và thêm comment cho các class đã hiểu.
- Dùng navigation history để giữ tuyến truy vết; ghi lại class.method và bằng chứng thay vì chỉ chụp màn hình.

7.2 Đi từ UI đến logic

Với XML layout, tìm ID hoặc text của nút trong res/layout và strings.xml, rồi search R.id.<name> hoặc findViewById, ViewBinding/DataBinding. Với Jetpack Compose, tìm chuỗi hiển thị và onClick lambda. Với app obfuscate, text hiển thị, resource ID và API framework thường ổn định hơn tên method.

Chuỗi truy vết thực tế

Màn hình -> chuỗi "Sai mã PIN"
-> find usage của resource/string
-> nhánh hiển thị Toast hoặc TextView
-> điều kiện ngay trước nhánh
-> method tạo giá trị điều kiện
-> input và dữ liệu so sánh

7.3 Kiểm tra Manifest theo mục tiêu bảo mật

Mục	Đọc gì	Không được kết luận vội
Exported component	caller validation, permission, input	exported không đồng nghĩa exploitable
Deep link	scheme host path, canonicalization	có BROWSABLE không đồng nghĩa open redirect
allowBackup	target SDK, device policy, dữ liệu	true không luôn cho phép backup thực tế
cleartext	network security config, endpoint	false không chứng minh không có traffic khác
debuggable	manifest cuối và runtime	source Gradle không phải manifest đã đóng gói

7.4 Tìm dữ liệu nhạy cảm và crypto

Search API và luồng dữ liệu thay vì chỉ search từ secret. Với SharedPreferences, database và file, xác định dữ liệu ghi vào có nhạy cảm không, mode và vị trí. Với crypto, ghi thuật toán, mode, IV/nonce, key source, vòng đời key và mục đích. Hardcoded key trong app có thể bị lấy; Base64 không phải mã hóa; hash không thay thế encryption.

- Cipher.getInstance, SecretKeySpec, IvParameterSpec, MessageDigest, Mac và AndroidKeyStore.
- SharedPreferences.Editor.putString, SQLiteDatabase, Room, FileOutputStream và cache.
- Log.d/e, Timber, printStackTrace và analytics events có thể làm lộ dữ liệu trong lab.
- WebView addJavaScriptInterface, setJavaScriptEnabled, loadUrl và shouldOverrideUrlLoading cần xem cùng nguồn URL/content.

7.5 Apktool và Smali

Decode và build

```
apktool d app.apk -o work/app_decoded
apktool b work/app_decoded -o output/app-unsigned.apk
```

Apktool phù hợp khi cần Manifest/resource gần dạng XML, xem Smali theo DEX hoặc rebuild lab. Nó không nhằm tạo Java source đẹp. Trong thư mục smali_classes2, smali_classes3, số thứ tự tương ứng các DEX bổ sung. Luôn search descriptor Lcom/example/Class;->method(Args)Return khi đối chiếu với JADX.

7.6 Tự động hóa không thay thế suy luận

MobSF, APKID và scanner khác hữu ích để inventory, tìm rule phổ biến và tạo điểm xuất phát. Kết quả scanner là giả thuyết. Một URL, permission hoặc hàm crypto không tự chứng minh có lỗi hổng. Báo cáo tốt phải có đường gọi, input kiểm soát được, điều kiện, tác động và thử nghiệm tái hiện.

8 Phân tích động bằng adb logcat proxy và Frida

8.1 adb là dây thần kinh của lab

Các lệnh nền tảng

```
adb devices -l
adb install app.apk
adb shell pm list packages | findstr example
adb shell dumpsys package com.example.app
adb shell am force-stop com.example.app
adb shell monkey -p com.example.app 1
adb logcat --pid=$(adb shell pidof com.example.app)
```

Trên PowerShell/CMD, cú pháp command substitution dòng cuối không giống Bash; có thể lấy PID trước rồi truyền vào adb logcat --pid <PID>. dumpsys package cho biết component, permission,

path APK và trạng thái cài. am start cho phép gọi Activity lab bằng explicit component hoặc URI để kiểm tra deep link.

Khởi chạy component và lấy APK được phép

```
adb shell am start -n com.example.app/.MainActivity
adb shell am start -a android.intent.action.VIEW -d "demo://open/item/42"
adb shell pm path com.example.app
adb pull /data/app/.../base.apk
```

8.2 Logcat và trạng thái process

- Xóa log trước thử nghiệm nếu cần: adb logcat -c; ghi thời điểm bấm nút hoặc gửi Intent.
- Lọc theo PID, tag, exception hoặc package; process có thể restart nên PID thay đổi.
- Dùng dumsys activity top/activities để biết Activity hiện tại; dumsys window cũng có thể hỗ trợ.
- Không coi thiếu log là thiếu hành vi: release build có thể loại log, hoặc code chạy ở process khác.

8.3 Quan sát traffic

Proxy như Burp Suite hoặc mitmproxy cho phép xem request/response nếu app chấp nhận CA của lab và traffic đi qua proxy. Android hiện đại phân biệt system CA và user CA; app có thể cấu hình Network Security Config hoặc certificate pinning. Hãy thực hành trên app lab. Nếu HTTPS không hiện, xác định lần lượt: DNS/kết nối, proxy, trust store, network config, pinning, HTTP/3 hoặc native stack.

Nguyên tắc. Mục tiêu là kiểm chứng data flow của ứng dụng được phép. Không thu thập token hay dữ liệu của người khác; dùng tài khoản test và server test.

8.4 Frida theo cách dễ hiểu

Frida chèn instrumentation vào process để quan sát hoặc thay đổi lời gọi khi chạy. Với Java layer, Java.perform đợi VM sẵn sàng; Java.use lấy wrapper của class; implementation thay method. Đây là cách kiểm chứng tham số và return, không phải phép màu tự hiểu app.

Hook quan sát method trong app lab

```
Java.perform(function () {
  const Checker = Java.use('com.example.lab.PinChecker');
  Checker.check.overload('java.lang.String').implementation = function (pin) {
    const result = this.check(pin);
    console.log('check pin length=' + pin.length + ' result=' + result);
    return result; // chỉ quan sát, không đổi hành vi
  };
});
```

Liệt kê app và spawn với script

```
frida-ps -Uai
frida -U -f com.example.lab -l scripts/trace.js
```

Nếu overload sai, class chưa load hoặc method bị inlining/native hóa, hook có thể không chạy. Kiểm tra class name đầy đủ, overload signature, process phụ, class loader và thời điểm attach. Với app không root, Frida Gadget là một hướng trên app bạn có quyền sửa; người mới nên dùng emulator/rooted lab để giảm biến số.

8.5 Phương pháp event đến code

Đây là kỹ thuật quan trọng nhất khi app lớn. Tạo một event rõ ràng, ví dụ bấm Login. Ghi network, log, Activity và thread. Search chuỗi UI/request endpoint. Hook một method gần event để xem có chạy không. Sau đó dịch dần hook lên caller hoặc xuống callee cho đến khi khoanh được logic. Mỗi bước phải có bằng chứng thời gian và input tương ứng.

9 Native code JNI và file so

9.1 Khi nào phải xuống native

- JADX thấy method có từ khóa native nhưng không có body.
- Code gọi System.loadLibrary hoặc ReLinker; APK có lib/<ABI>/*.so đáng kể.
- Chuỗi hoặc thuật toán quan trọng không xuất hiện trong DEX.
- App dùng Unity IL2CPP, Flutter, engine game, codec hoặc thư viện crypto native.

9.2 Từ Java method tới hàm native

JNI có thể map theo tên export Java_package_Class_method hoặc đăng ký động qua RegisterNatives trong JNI_OnLoad. Với đăng ký động, tìm JNI_OnLoad, RegisterNatives, bảng JNINativeMethod hoặc hook đăng ký ở runtime. Cần xác định đúng ABI mà emulator đang chạy.

Cầu nối Java và native

```
public native boolean verify(byte[] data);

// Tên export kiểu tĩnh có thể gần dạng:
Java_com_example_lab_Checker_verify
```

9.3 Quy trình native cơ bản

- Dùng file/readelf/objdump hoặc công cụ PE/ELF tương đương để xem ABI, section, dynamic symbols và imports.
- Mở .so trong Ghidra/IDA; chọn kiến trúc đúng; chạy analysis; tìm strings, imports và JNI entry.
- Đổi tên hàm/biến theo vai trò đã chứng minh; dựng struct và prototype dần dần.
- Đối chiếu với hook native của Frida hoặc debugger; ASLR khiến địa chỉ runtime khác offset file.

Đừng xuống native quá sớm. Nếu Java layer đã cho biết input, output và thời điểm gọi, hãy mang các mốc đó xuống Ghidra/IDA. Phân tích .so không có ngữ cảnh thường tốn thời gian hơn nhiều.

10 Sửa build và ký lại APK lab

10.1 Chuỗi thao tác

Build align sign verify install

```
apktool d lab.apk -o work/lab
# sửa resource hoặc Smali nhỏ trong work/lab
apktool b work/lab -o output/lab-unsigned.apk

zipalign -p -f 4 output/lab-unsigned.apk output/lab-aligned.apk

apksigner sign --ks lab.keystore --out output/lab-signed.apk output/lab-aligned.apk
apksigner verify --verbose --print-certs output/lab-signed.apk
adb install output/lab-signed.apk
```

Nếu chưa có lab keystore, tạo một khóa riêng chỉ cho thực hành. Không dùng production key. Sau khi ký bằng key khác, Android không cho update bản gốc. Gỡ bản cũ sẽ xóa private app data trừ trường hợp backup/restore hoặc cấu hình khác; chỉ làm trên emulator lab.

10.2 Sửa branch Smali có kiểm soát

Trong crackme, một bài tập hợp lệ là thay nhánh hoặc giá trị trả về để quan sát hậu quả. Trước khi sửa, ghi register type, đường đi thành công/thất bại, try/catch và side effect. Sau khi sửa, diff Smali, build, verify, chạy test cả đường đúng và sai. Nếu chỉ ép UI hiển thị thành công nhưng server vẫn từ chối, bạn mới chỉ thay presentation state.

Ví dụ minh họa trên crackme do bạn được phép sửa

```
# Trước: return kết quả tính trong v0
return v0

# Bài tập lab: đặt hằng boolean rồi return
const/4 v0, 0x1
return v0
```

10.3 Lỗi thường gặp

Triệu chứng	Nguyên nhân có thể	Cách kiểm tra
INSTALL_FAILED_UPDATE_INCOMPATIBLE	Khác signer	Gỡ bản lab cũ hoặc dùng cùng test key
Parse error	APK hỏng hoặc ký/align sai	apksigner verify và aapt2 dump badging
Resource build error	XML/resource decode không round-trip	Đọc dòng lỗi Apktool, framework phù hợp

Triệu chứng	Nguyên nhân có thể	Cách kiểm tra
App crash lúc mở	Smali sai register/type hoặc integrity check	logcat exception và stack trace
Thiếu native library	Chỉ cài base APK hoặc sai ABI	pm path, splits và ro.product.cpu.abi.list

11 Obfuscation packing và anti-analysis

11.1 Obfuscation thay hình thức không xóa hành vi

Kỹ thuật	Dấu hiệu	Cách tiếp cận
Rename	class a.a.a, method a b	resource, API, type, xref và runtime
String encryption	string tạo lúc chạy	hook decryptor hoặc điểm dừng chuỗi
Reflection	Class.forName, Method.invoke	log class/method name tại runtime
Control-flow transform	branch và exception rối	Smali CFG, slice theo data, dynamic trace
Dynamic DEX	DexClassLoader, code_cache	xác định nguồn, hash, dump trong lab
Native chuyển logic	native method, .so lớn	JNI map và native analysis

11.2 Anti-debug anti-root anti-emulator

App có thể kiểm tra `Debug.isDebuggerConnected`, `TracerPid`, `package/tool`, `file su`, `build tags`, `sensor`, `device property`, `timing`, `port` hoặc `integrity`. Không công cụ nào bypass hoàn toàn mọi cơ chế. ScyllaHide là công cụ Windows và không giải quyết toàn bộ anti-analysis Android. Frida script công khai cũng có thể bị nhận diện hoặc không phù hợp phiên bản.

- Trước tiên xác định check nào thực sự gây nhánh thoát, không patch toàn bộ danh sách dấu hiệu.
- Phân biệt detection với response: check có thể thu telemetry nhưng không block ngay.
- Hook hoặc patch ở điểm quyết định chung thường ổn định hơn và hàng chục API rời rạc trong lab.
- Nếu quyết định nằm server side hoặc dùng Play Integrity với backend, sửa local không tự tạo kết quả hợp lệ từ server.

11.3 Packer và code tải động

Packer có thể để DEX nhỏ làm loader, giải mã code khi chạy và thay class loader. Dấu hiệu gồm application class lạ, rất ít logic trong DEX, asset blob lớn, native library chịu trách nhiệm load và class chỉ xuất hiện runtime. Với mẫu được phép, mục tiêu là xác định thời điểm code rõ xuất hiện trong memory/class loader và thu artifact để phân tích tiếp, đồng thời giữ hash và nhật ký nguồn gốc.

12 Bài thực hành từ đầu đến cuối

12.1 Mục tiêu lab

Dùng một crackme Android hợp pháp có màn hình nhập mã. Mục tiêu không phải đoán PIN nhanh nhất, mà là tạo báo cáo chứng minh: input đi qua method nào, so sánh local hay server, dữ liệu bí mật nằm ở đâu, branch thành công được kích hoạt thế nào và decompiler có sai lệch gì.

12.2 Các bước bắt buộc

1. Tạo snapshot emulator và thư mục case; tính SHA-256 APK.
2. Chạy apksigner verify, aapt2 dump badging và liệt kê file/ABI.
3. Mở Manifest, ghi launcher Activity, application class, exported components và network config.
4. Chạy app gốc; nhập hai giá trị khác nhau; lưu log, traffic và ảnh trạng thái.
5. Trong JADX, đi từ chuỗi lỗi hoặc ID nút đến callback; vẽ Source -> Transform -> Sink.
6. Kiểm tra method quyết định bằng Smali; ghi descriptor và instruction branch.
7. Hook method trên lab để log độ dài input và return, tránh in PIN thật vào báo cáo.
8. Nếu bài yêu cầu patch, sửa một điểm nhỏ; build, align, ký và verify.
9. Test regression: input sai, input đúng, restart, offline/online và process mới.
10. Viết kết luận tách rõ quan sát, suy luận và điều chưa chứng minh.

12.3 Mẫu báo cáo một trang

Mục	Nội dung cần ghi
Mục tiêu	Câu hỏi hẹp và phạm vi được phép
Artifact	Tên, SHA-256, package, version, signer digest
Bản đồ	Component, entry point, method descriptor, native library nếu có
Bảng chứng tĩnh	Đường gọi, data flow, Smali quan trọng
Bảng chứng động	Lệnh chạy, input test, log/traffic/hook result
Kết luận	Điều đã chứng minh, mức tin cậy, giới hạn

Mục	Nội dung cần ghi
Tái hiện	Các bước ngắn để người khác kiểm tra lại

12.4 Tiêu chí tự chấm

Mức	Bạn làm được gì
1 Nhận diện	Mở APK, biết file chính và launcher component
2 Định hướng	Từ UI đi đến callback và method nghỉ ngơi
3 Chứng minh	Kết hợp Smali với quan sát runtime
4 Can thiệp lab	Sửa nhỏ, rebuild, sign và regression test
5 Giải thích	Viết được data flow, giới hạn và lý do kết luận

13 Lộ trình học sáu tuần

Tuần	Trọng tâm	Sản phẩm phải nộp cho chính mình
1	Android component, Manifest, Intent, lifecycle, adb	Bản đồ một app tự viết và 20 lệnh adb có chú thích
2	APK, resource, DEX, JADX	Phân tích tĩnh app mã nguồn mở rồi đối chiếu source
3	Smali, branch, xref, chữ ký	Patch một crackme đơn giản và giải thích từng instruction đối
4	Logcat, proxy, Frida Java	Trace một event từ UI đến network hoặc storage
5	JNI, ELF, Ghidra/IDA cơ bản	Map một native method và xác nhận input/output runtime
6	Obfuscation và case hoàn chỉnh	Writeup có hash, bằng chứng tĩnh/động, giới hạn

Mỗi tuần chọn một mục tiêu nhỏ và hoàn tất writeup trước khi chuyển bài. Không cần học hết Android development; cần đủ khả năng đọc component, callback, collection, coroutine cơ bản, network/storage API và build một app đồ chơi để hiểu phía compiler tạo ra gì.

13.1 Cách rút kỹ thuật từ một bài

- **Gọi tên cơ chế:** ví dụ reflection, dynamic DEX, JNI registration, certificate pinning.
- **Ghi dấu hiệu nhận biết:** API, file, log, control-flow hoặc runtime artifact nào dẫn tới nhận diện.
- **Ghi nguyên lý:** cơ chế hoạt động ở lớp nào và bảo vệ điều gì.
- **Ghi điểm quan sát:** entry point, chokepoint, input/output và công cụ kiểm chứng.
- **Tạo biến thể:** tự sửa app lab để cơ chế đổi tên, chuyển sang native hoặc chạy ở process khác.
- **Viết giới hạn:** trường hợp kỹ thuật thất bại; dấu hiệu nào là false positive.

Không học đệ quy vô hạn. Khi gặp khái niệm lạ, chỉ học đủ để trả lời ba câu: nó nằm ở lớp nào, ảnh hưởng đường phân tích nào, và bằng chứng nào xác nhận. Quay lại bài; đào sâu sau khi cơ chế lặp lại ở nhiều case.

14 Cheat sheet xử lý lỗi và tài liệu tham khảo

14.1 Cheat sheet lệnh

Bảng lệnh tối thiểu

```
# Nhận diện
certutil -hashfile app.apk SHA256
aapt2 dump badging app.apk
apksigner verify --verbose --print-certs app.apk

# Thiết bị
adb devices -l
adb shell pm path com.example.app
adb shell dumpsys package com.example.app
adb shell pidof com.example.app
adb logcat

# Static
jadx-gui app.apk
apktool d app.apk -o work/app

# Rebuild lab
apktool b work/app -o output/app-unsigned.apk
zipalign -p -f 4 output/app-unsigned.apk output/app-aligned.apk
apksigner sign --ks lab.keystore --out output/app-signed.apk output/app-aligned.apk
apksigner verify --verbose output/app-signed.apk
```

14.2 Cây xử lý khi không tìm thấy logic

- Search string không thấy: kiểm tra resource, mã hóa string, ngôn ngữ khác, asset, native hoặc tải từ server.
- JADX chỉ thấy loader: tìm DexClassLoader, Application class, asset blob và native load.
- Hook không chạy: kiểm tra process, class loader, overload, timing, inlining và logic native.
- Traffic trống: kiểm tra proxy/trust, QUIC, native networking, pinning và app có thực sự gửi request không.
- Bản ký lại crash: đọc logcat; kiểm tra signature/integrity, resource, multidex, split và ABI.

- Code quá lớn: quay lại event -> string/resource -> callback -> data flow; bỏ qua library không liên quan.

14.3 Thuật ngữ nhanh

Thuật ngữ	Giải thích ngắn
Decompile	Dựng code bậc cao gần Java từ bytecode; có thể sai hoặc mất thông tin
Disassemble	Chuyển bytecode/machine code thành instruction gần nguyên bản
Smali	Assembly syntax cho DEX
ART	Android Runtime thực thi DEX với cơ chế interpreter/JIT/AOT tùy phiên bản
ABI	Quy ước nhị phân theo kiến trúc như arm64-v8a, armeabi-v7a, x86_64
JNI	Cầu nối Java/Kotlin với native C/C++
Obfuscation	Biến đổi làm khó hiểu nhưng giữ hành vi
Packing	Đóng gói/ẩn code và giải nạp khi chạy
Hook	Chặn lời gọi để quan sát hoặc thay đổi runtime
Xref	Tham chiếu tới/từ một symbol, method, field hoặc string

14.4 Tài liệu chính thức và lab

1. [Android Developers - Application fundamentals](#) - Component, sandbox, Manifest và resource.
2. [Android Developers - apksigner](#) - Ký và xác minh APK.
3. [Android Open Source Project - Dalvik bytecode format](#) - Instruction và định dạng bytecode.
4. [JADX project](#) - Decompiler DEX/APK và hướng dẫn sử dụng.
5. [Apktool documentation](#) - Decode, build và resource framework.
6. [Frida Android tutorial](#) - Thiết lập và dynamic instrumentation.
7. [OWASP Mobile Application Security](#) - MASTG, MASVS, kỹ thuật và crackmes.
8. [MobSF documentation](#) - Static/dynamic analysis tự động hóa.
9. [Android Debug Bridge](#) - Cú pháp và khả năng adb.
10. [Android app signing](#) - Mô hình khóa ký và phát hành.

14.5 Checklist trước khi kết luận

- Tôi có hash và biết chính xác artifact/version đang phân tích.
- Tôi đã phân biệt APK đơn, split APK và AAB/container.
- Tôi đã xác định stack: DEX, native, Flutter, Unity, React Native hoặc WebView.
- Tôi có entry point và đường data flow, không chỉ một từ khóa trùng khớp.
- Tôi đã kiểm tra bytecode hoặc runtime tại điểm decompiler có thể sai.
- Tôi tách quan sát khỏi suy luận và ghi mức tin cậy.
- Mọi thử nghiệm diễn ra trong phạm vi được phép, với dữ liệu test.
- Người khác có thể tái hiện kết quả từ lệnh, input và điều kiện tôi ghi.

Kết lời

Reverse APK không phải là bấm một nút để lấy lại source. Đó là quá trình ghép nhiều biểu diễn: Manifest cho biết hệ thống có thể đi vào đâu; resource nối giao diện với code; DEX/Smali mô tả logic; native library che giấu một phần khác; runtime cho biết đường nào thực sự chạy. Khi duy trì câu hỏi hẹp, bản đồ rõ và nhật ký bằng chứng, app hàng nghìn class sẽ trở thành một tập các tuyến truy vết nhỏ có thể giải quyết lần lượt.